

Лабораторная работа №8

**Реализация основных моделей в дискретно-событийном
подходе**

Люпп Софья Романовна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	9
5	Выводы	19
	Список литературы	20

Список иллюстраций

4.1	sir_des.jl	9
4.2	Бета=0.03	10
4.3	Бета=0.05	10
4.4	Бета=0.07	11
4.5	sir_des.jl с фикс величиной	11
4.6	Бета=0.03 с фикс величиной	12
4.7	Бета=0.05 с фикс величиной	12
4.8	Бета=0.07 с фикс величиной	13
4.9	Бенчмарк	13
4.10	Бенчмарк	13
4.11	csv	14
4.12	csv	15
4.13	Модель SIR с демографией	16
4.14	Модель SIR с вакцинацией	16
4.15	Анализ	17
4.16	срабатывание события вакцинации по таймеру	17
4.17	Латентный период	18

Список таблиц

1 Цель работы

Изучить дискретно-событийный подход к имитационному моделированию на примере классической модели распространения инфекции SIR.

2 Задание

- 1) Изучить теорию по дискретно-событийному моделированию;
- 2) Создать необходимые скрипты по заданию;
- 3) Выполнить дополнительные задания;
- 4) Выполнить отчет и презентацию, преобразовать документацию в формате Quarto и выполнить литературный код.

3 Теоретическое введение

Программный комплекс состоит из двух основных файлов: — `src/sir_model.jl` (ядро модели): определение структур данных, функций событий, логики агентов и управления симуляцией. — `scripts/sir_des.jl` (скрипт запуска): задание параметров, инициализация модели, выполнение прогона и визуализация.

Используемые внешние библиотеки — `ResumableFunctions`: создание возобновляемых функций-генераторов с помощью макроса **resumable** и оператора **yield**. — `ConcurrentSim`: ядро дискретно-событийного моделирования: управление виртуальным временем, планирование событий через `timeout` и запуск процессов через **process**. — `Distributions`: генерация случайных величин (экспоненциальное, дискретное равномерное, равномерное распределения). — `DataFrames`: формирование таблицы результатов. — `StatsPlots`: построение графиков.

Вспомогательные функции мутации массивов статистики — `increment!(a::Array{Int64})` — добавляет в конец массива значение, увеличенное на 1 относительно последнего элемента. — `decrement!(a::Array{Int64})` — добавляет значение, уменьшенное на 1. — `carryover!(a::Array{Int64})` — дублирует последнее значение. Эти функции используются для фиксации изменений численности S, I, R в момент наступления события. Массивы `Sa`, `Ia`, `Ra` хранят временные ряды численности: каждая операция добавляет новую точку, соответствующую текущему модельно времени.

Функция `live` описывает жизненный цикл индивида. Она размечена макросом **resumable**, что позволяет приостанавливать её выполнение с помощью **yield** `timeout(...)`, не блокируя поток. Логика работы: — Пока индивид находится в

состоянии :S: — Ожидает случайное время, распределённое экспоненциально с параметром $1/c$. Это моделирует промежуток до следующего социального контакта. — После ожидания выбирает случайного собеседника (alter) из всей популяции (исключая себя). — Если собеседник в состоянии :I, то с вероятностью β происходит заражение: статус меняется на :I и вызывается `infection_update!`. — Если индивид стал инфицированным (:I): — Ожидает время до выздоровления, распределённое экспоненциально с параметром $1/\gamma$. — После этого статус меняется на :R, и вызывается `recovery_update!`.

4 Выполнение лабораторной работы

Начинаю лабораторную с главного скрипта, где будет реализована вычислительная логика модели SIR в src, а затем делаю скрипт в scripts `sir_des.jl`. Вывожу полученные результаты (рис. 4.1).

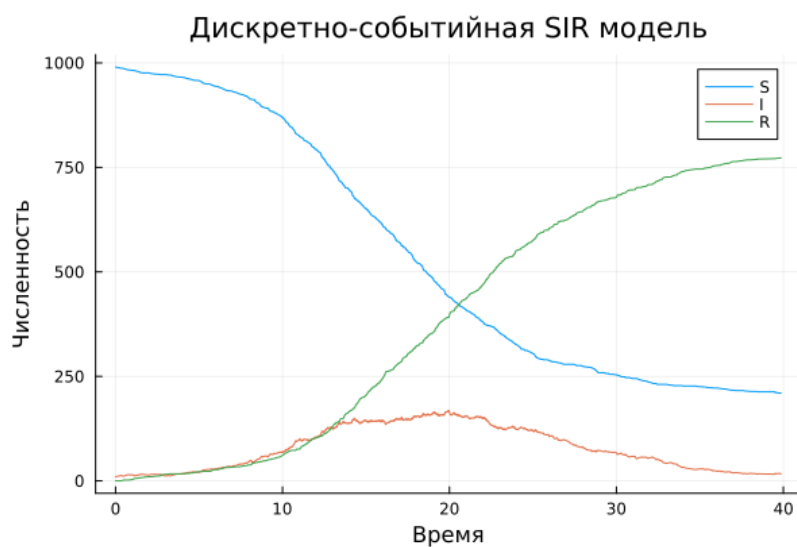


Рисунок 4.1: `sir_des.jl`

После этого хочу проанализировать цикл, варьирующий параметры бета, получаю три результата для трех разных значений (рис. 4.2) (рис. 4.3) (рис. 4.4) .

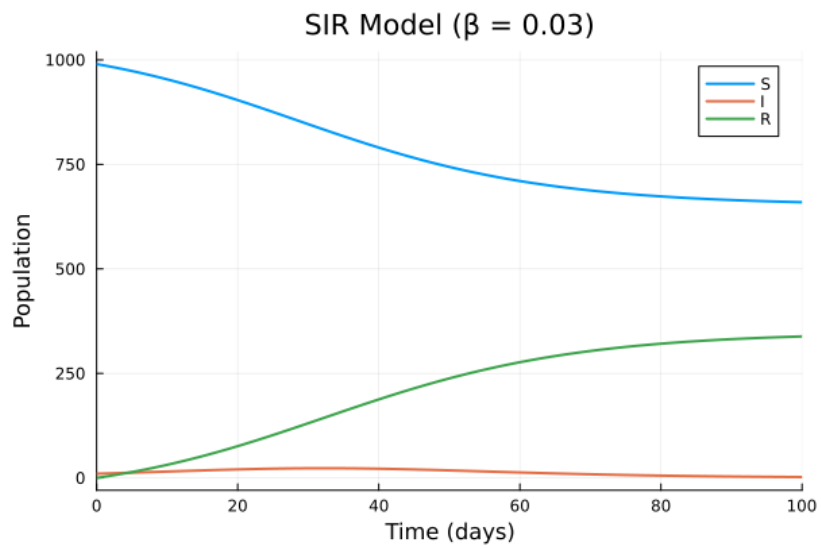


Рисунок 4.2: Бета=0.03

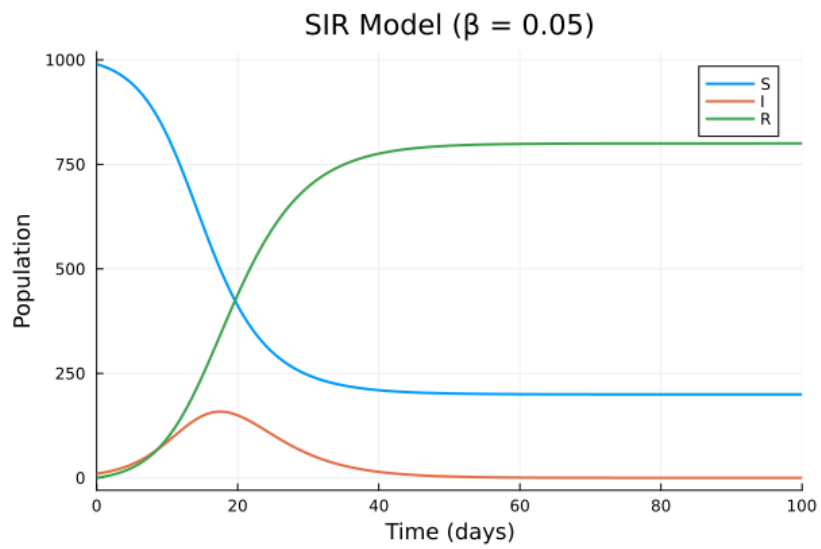


Рисунок 4.3: Бета=0.05

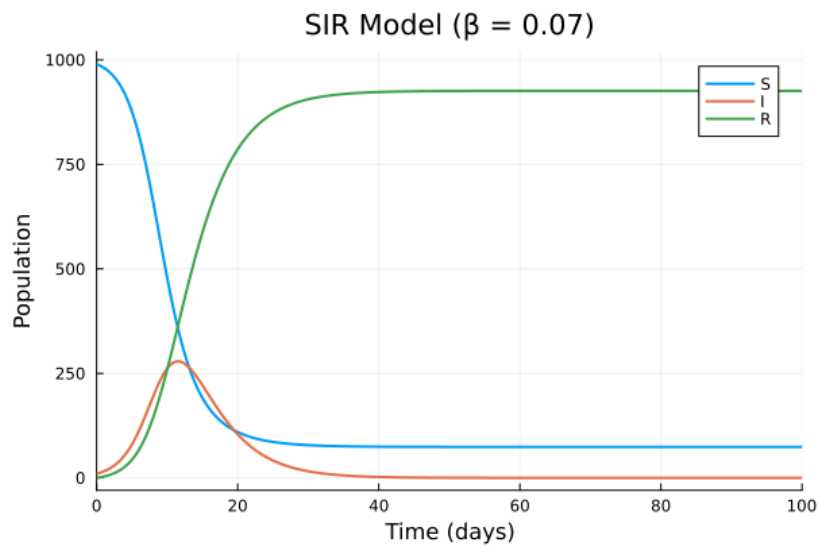


Рисунок 4.4: Бета=0.07

Заменяю экспоненциальное время выздоровления на фиксированную величину $1/\gamma$ и получаю результаты для графиков `sir_des.jl` и графиков с различными значениями бета и обращаю внимание на более синхронное выздоровление (рис. 4.5) (рис. 4.6) (рис. 4.7) (рис. 4.8).

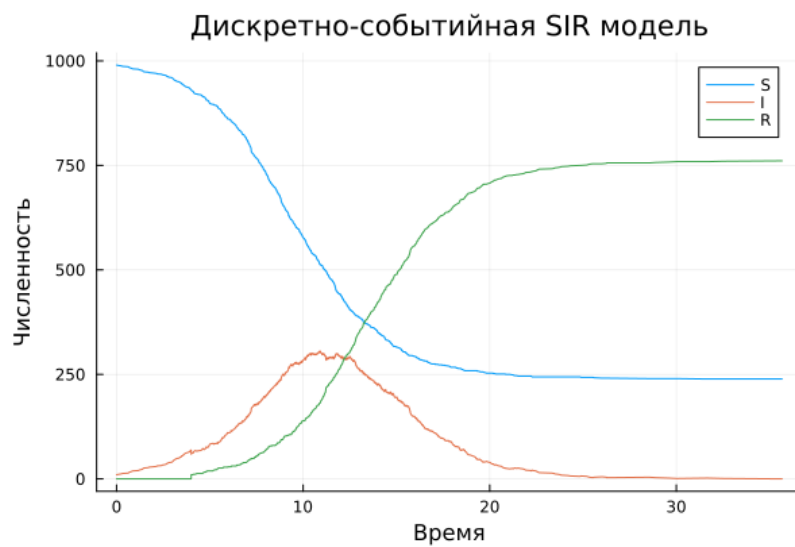


Рисунок 4.5: `sir_des.jl` с фикс величиной

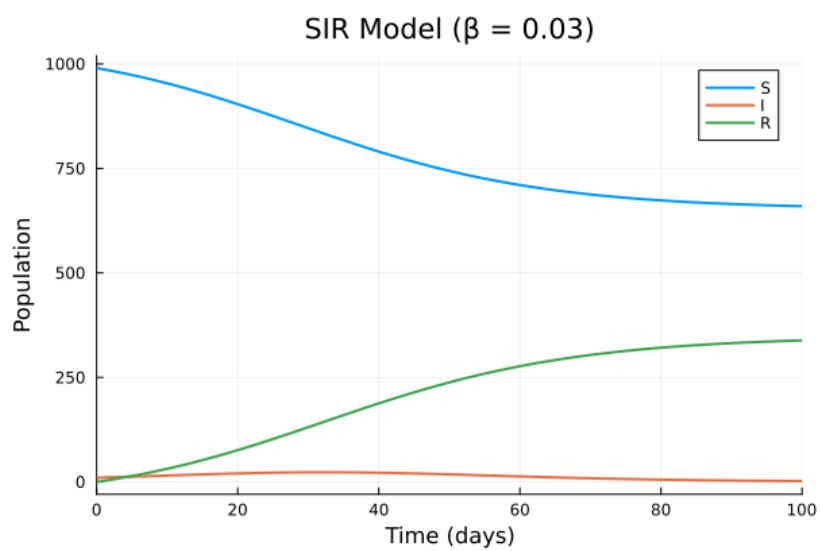


Рисунок 4.6: Бета=0.03 с фикс величиной

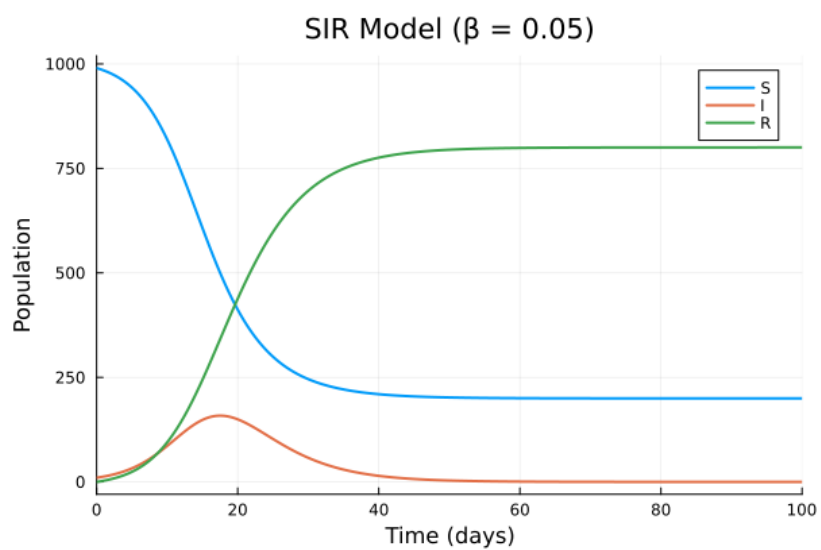


Рисунок 4.7: Бета=0.05 с фикс величиной

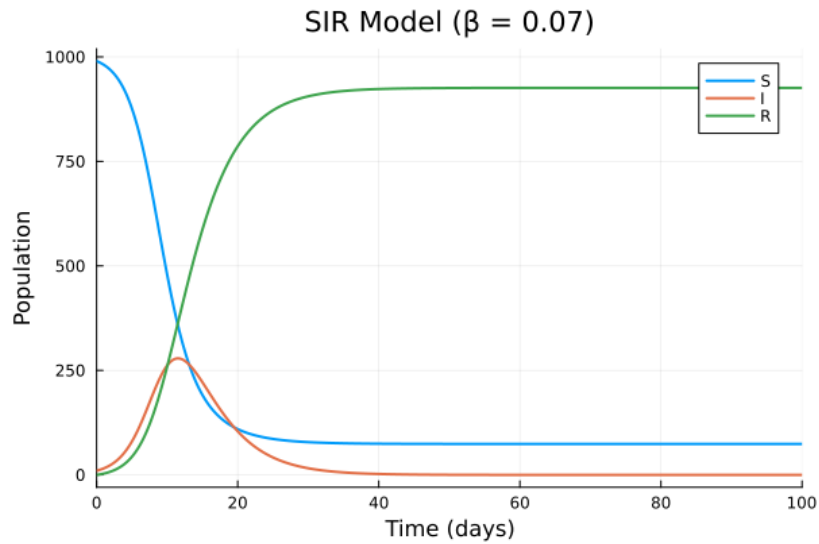


Рисунок 4.8: Бета=0.07 с фикс величиной

С помощью макроса **benchmark** измеряю время выполнения `sir_run` для популяции 10000 индивидов с определенным способом оптимизации. Полученные результаты отображены в консоль (рис. 4.9) (рис. 4.10).

```
y--simulationmodeling/labs/lab08/project130$ julia --project=. scripts/sir_run.jl
l

=== Бенчмарк для популяции 10000 ===
Медианное время выполнения: 1.130489e6 секунд
Минимальное время: 1.074521e6 секунд
Максимальное время: 4.319162e6 секунд
Среднее время: 1.1698424596698114e6 секунд

Результат моделирования:
```

Рисунок 4.9: Бенчмарк

```
ion-modeling/2026-1--study--simulationmodeling/labs/lab08/project/scripts/sir_run.jl:64
srluipp@ybox:~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--stud
y--simulationmodeling/labs/lab08/project1$ julia --project=. scripts/sir_run.jl
Бенчмарк модели SIR
Время выполнения: 2.337666666666667e-6 секунд
Среднее время: 3.3520575888888895e-6 секунд
Минимальное время: 2.337666666666667e-6 секунд
Максимальное время: 0.0005099435555555556 секунд
Количество вызовов: 10000
```

Рисунок 4.10: Бенчмарк

Сохраняю результаты в формате csv (рис. 4.11) (рис. 4.12).

Text Import - [sir_N10000_b0.3_g0.1_t100.0_2026-05-30_15-27.csv]

Import

Character set: Unicode (UTF-8)

Locale: Default - English (USA)

From row: 1

Separator Options

☐ Fixed width ☒ Separated by

☒ Tab ☒ Comma ☒ Semicolon ☐ Space ☐ Other

☐ Merge delimiters ☐ Trim spaces String delimiter: "

Other Options

☐ Format quoted field as text ☐ Detect special numbers

☐ Evaluate formulas ☒ Detect scientific notation

Fields

Column type:

	Standard	Standard	Standard	Standard
1	time	S	I	R
2	0.0	9990.0	10.0	0.0
3	1.0	9987.003	11.997	1.1997
4	2.0	9983.408577750271	14.39172224973	2.6388722249730003
5	3.0	9979.098224449575	17.262903325453617	4.365162557518362
6	4.0	9973.930178211858	20.70465923062462	6.435628480580824
7	5.0	9967.73497344596	24.82939807345975	8.9185682879268
8	6.0	9960.310187659567	29.77124405250687	11.895692693177487
9	7.0	9951.414262896502	35.69004441032069	15.464697134209556
10	8.0	9940.750270288854	42.77602247802686	19.74220028210224

Help Cancel OK

Рисунок 4.11: csv

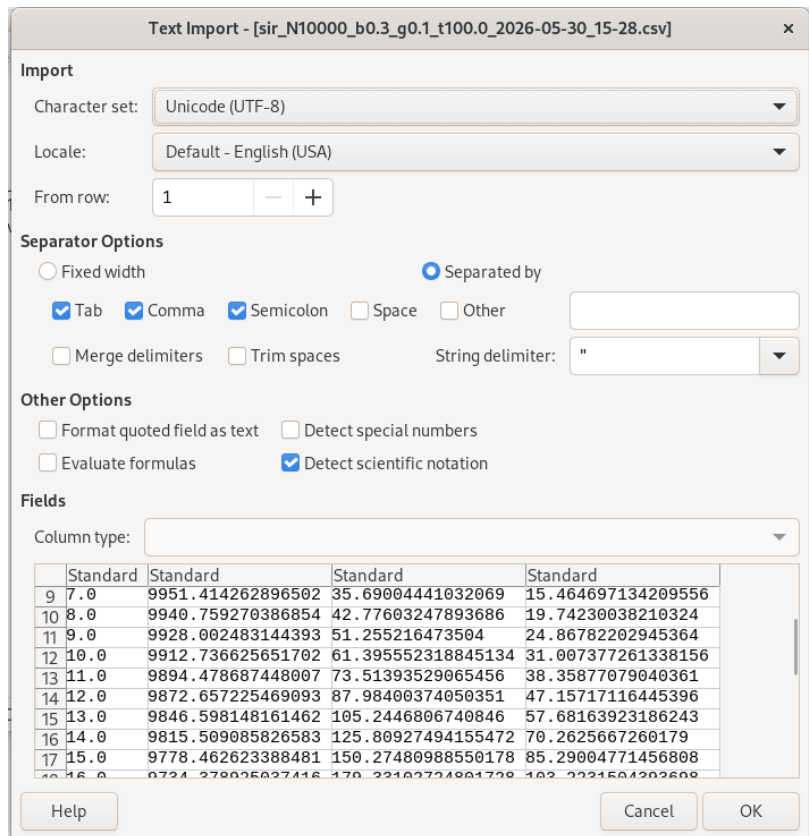


Рисунок 4.12: csv

После чего по заданию добавляю демографические события: расширяю модель, добавив в жизненный цикл индивида возможность смерти (с постоянной интенсивностью μ) и рождение новых восприимчивых. Вывожу график (рис. 4.13).

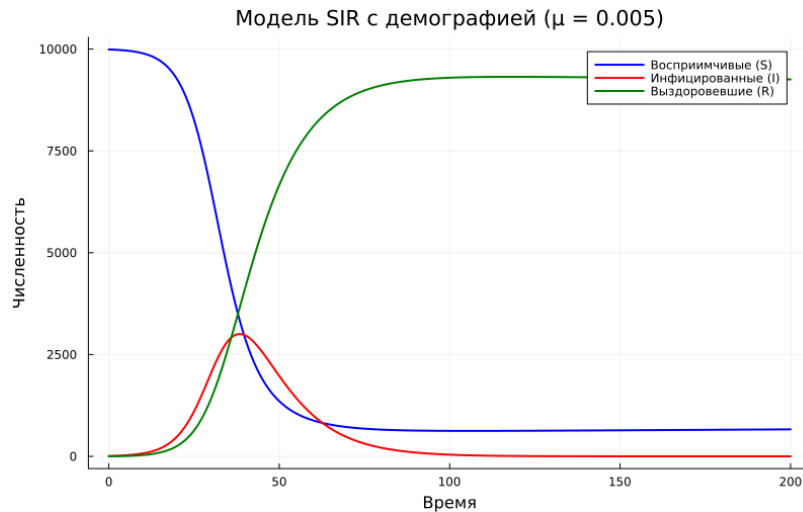


Рисунок 4.13: Модель SIR с демографией

В функцию live добавить блок ожидания смерти. Реализовываю стратегию вакцинации: в определённый момент времени (или при превышении порога инфицированных) часть восприимчивых мгновенно переводится в :R. Исследую, какая доля вакцинированных необходима для предотвращения эпидемии (рис. 4.14) (рис. 4.15).

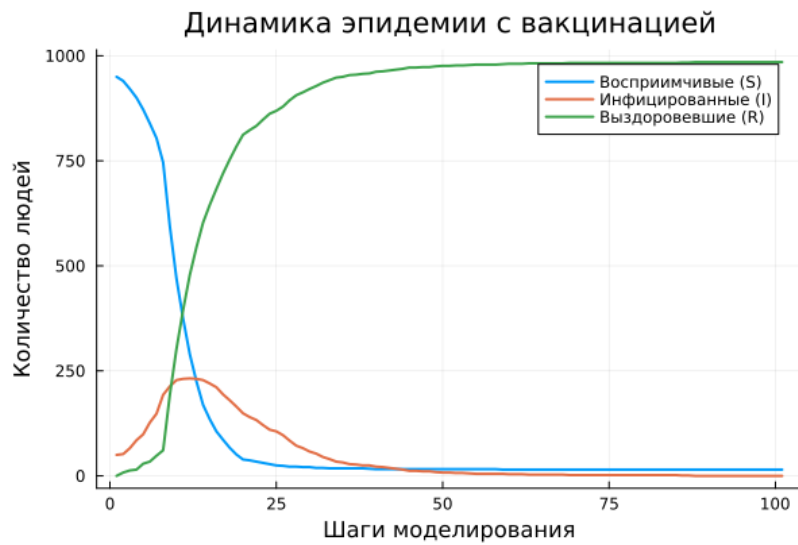


Рисунок 4.14: Модель SIR с вакцинацией


```

y--simulationmodeling/labs/lab08/project1$ julia --project=. scripts/vaccination
.jl
Шаг 1: S=940, I=52, R=8 (Доля инфицированных: 0.052)
Шаг 2: S=921, I=66, R=13 (Доля инфицированных: 0.066)
Шаг 3: S=900, I=85, R=15 (Доля инфицированных: 0.085)
Шаг 4: S=872, I=99, R=29 (Доля инфицированных: 0.099)
Шаг 5: S=839, I=127, R=34 (Доля инфицированных: 0.127)
Шаг 6: S=804, I=148, R=48 (Доля инфицированных: 0.148)
Вакцинация на шаге 7: инфицированных 0.193, привито 112 агентов.
Шаг 7: S=634, I=193, R=173 (Доля инфицированных: 0.193)
Вакцинация на шаге 8: инфицированных 0.213, привито 89 агентов.
Шаг 8: S=506, I=213, R=281 (Доля инфицированных: 0.213)
Вакцинация на шаге 9: инфицированных 0.228, привито 70 агентов.
Шаг 9: S=400, I=228, R=372 (Доля инфицированных: 0.228)
Вакцинация на шаге 10: инфицированных 0.231, привито 56 агентов.
Шаг 10: S=318, I=231, R=451 (Доля инфицированных: 0.231)
Вакцинация на шаге 11: инфицированных 0.232, привито 44 агентов.
Шаг 11: S=246, I=232, R=522 (Доля инфицированных: 0.232)
Вакцинация на шаге 12: инфицированных 0.231, привито 34 агентов.
Шаг 12: S=191, I=231, R=578 (Доля инфицированных: 0.231)
Вакцинация на шаге 13: инфицированных 0.228, привито 25 агентов.

```

Рисунок 4.15: Анализ

Добавляю в модель срабатывание события вакцинации по таймеру (рис. 4.16).

```

mutable struct SIRMModel
    agents::Vector{Agent}
    beta::Float64          # Темп инфицирования
    gamma::Float64         # Темп выздоровления
    vaccination_fraction::Float64 # Доля вакцинируемых из :S
    vaccination_timer::Int  # Периодичность вакцинации (шаги)
    next_vaccination_step::Int # Следующий шаг вакцинации
    history::Vector{Tuple{Int, Int, Int}} # История (S, I, R)
    time_step::Int
end

```

Рисунок 4.16: срабатывание события вакцинации по таймеру

Ввожу латентный период (статус :E). После заражения индивид переходит в :E и через время, распределённое экспоненциально с параметром σ , становится инфекционным (:I). Оцениваю, как меняется динамика по сравнению с SIR (рис. 4.17).

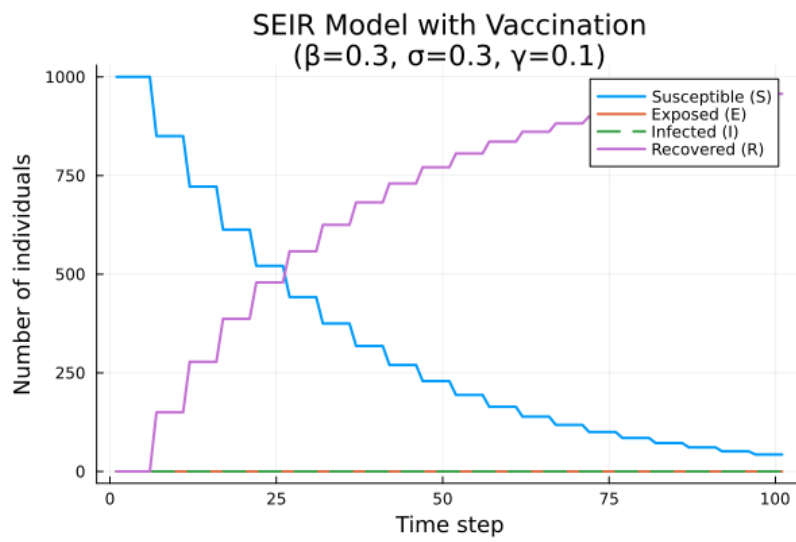


Рисунок 4.17: Латентный период

5 Выводы

В ходе лабораторной работы я изучила и реализовала основные модели в дискретно-событийном подходе (модель SiR)

Список литературы

- Имитационное моделирование. Практикум А. В. Королькова Кулябов Д. С.